

# Atelier noté – Pipeline CI avec GitHub Actions  
## Séance 2 – Bloc DevOps, ESIEA

### 8 étapes

---

## Application fournie

Le dossier `starter-app/` de ce répertoire contient une petite application Flask avec trois tests unitaires déjà écrits :

```
...
starter-app/
├─ app.py          # deux fonctions (alert_threshold, sanitize_input)
+ deux endpoints (/health, /status)
├─ test_app.py     # 3 tests déjà écrits : alert_threshold,
sanitize_input, /health
├─ requirements.txt # flask, pytest, pytest-cov, flake8
└─ .flake8         # config lint (max-line-length = 100)
...
```

Copiez ce dossier dans votre dépôt de séance 1 (à la racine, ou fusionnez avec ce qui existe déjà) – c'est le point de départ de tout l'atelier. Notez dès la découverte que `/status` n'a aucun test : vous en écrirez un à l'étape 4.

## Prérequis

Le dépôt Git de votre groupe créé en séance 1, avec au moins un membre ayant les droits d'administration (pour modifier la protection de branche à l'étape 7). Python 3.10+ installé localement pour tester avant de pousser :

```
```bash
cd starter-app
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pytest -v
flake8 . --max-line-length=100 --exclude=.venv
```
```

Les trois tests doivent passer et flake8 ne doit rien signaler avant même de commencer l'atelier – c'est la vérification de base que l'environnement est prêt.

---

## Étape 1 – Découverte de l'application fournie

### Consignes

Avant d'écrire le moindre workflow, comprenez ce que l'application vérifie déjà sans CI. Intégrez `starter-app/` dans votre dépôt de séance

1, installez l'environnement Python localement, et lisez le code de `app.py` et `test\_app.py` avant de lancer quoi que ce soit – cela évite de découvrir en CI un problème qui existait déjà en local. Exécutez tests et lint manuellement (`pytest`, `flake8`) : les trois tests fournis doivent passer, le lint ne doit rien signaler. Notez que `/status` n'a aucun test – vous le complerez à l'étape 4. Piège classique : modifier `app.py` à ce stade, alors que l'objectif est de comprendre l'existant, pas de le changer. Committez l'intégration comme un point de départ propre.

---

## Étape 2 – Premier workflow minimal

### Consignes

Créez votre premier fichier de workflow GitHub Actions, avec un unique job qui enchaîne : récupérer le code, installer Python, installer les dépendances, puis lancer les tests. Le fichier doit vivre dans `.github/workflows/` – un chemin mal orthographié fait que GitHub n'y voit rien, sans erreur explicite. Appuyez-vous sur des actions officielles toutes prêtes plutôt que de tout réécrire, notamment `actions/checkout` et `actions/setup-python`. Une fois poussé, vérifiez dans l'onglet **Actions** que le run apparaît et passe au vert, et ouvrez-le pour confirmer que les tests s'exécutent bien dans les logs.

---

## Étape 3 – Déclencheurs push & pull request

### Consignes

Vérifiez concrètement, pas seulement en théorie, que le workflow se déclenche aussi bien sur un push direct que sur une pull request : ouvrez une vraie PR de test et observez le check apparaître directement dans l'interface de la PR, avec son propre statut. Restreignez ensuite le déclencheur `push` à la branche `main` uniquement, pour ne pas gaspiller de minutes CI sur des branches de travail intermédiaires – attention à bien filtrer sur `branches` et non sur `paths`, qui filtre les fichiers modifiés plutôt que la branche cible. Une pull request doit continuer à déclencher le workflow normalement même après cette restriction.

---

## Étape 4 – Job de lint séparé

### Consignes

Séparez le job unique en deux jobs distincts, `lint` (flake8) et `test` (pytest), reliés par `needs:` pour que les tests n'attendent pas inutilement si le style est déjà en faute – sans ce lien, les deux jobs démarreraient en parallèle et l'intérêt du fail-fast disparaîtrait. Vérifiez concrètement l'effet : une faute de style volontaire doit faire échouer `lint` sans même lancer `test`. Ajoutez aussi le test manquant

pour l'endpoint `/status`, identifié comme absent à l'étape 1, en vous inspirant de la structure des tests déjà présents dans `test\_app.py`.

---

## Étape 5 – Matrix build

### Consignes

Paramétrez le job `test` pour qu'il s'exécute automatiquement sur trois versions de Python différentes, en parallèle, à partir d'une seule définition de job – c'est la notion de matrice (`strategy.matrix`) dans GitHub Actions. Piège classique : oublier de relier la version installée à la variable de matrice plutôt qu'à une valeur fixe, ce qui fait tourner trois jobs à l'apparence parallèle mais utilisant tous la même version en réalité.

---

## Étape 6 – Cache & artifacts

### Consignes

Ajoutez la mise en cache des dépendances pip pour accélérer les runs suivants, avec `actions/cache`, et conservez le rapport de couverture HTML en artefact téléchargeable via `actions/upload-artifact`, y compris quand les tests échouent (`if: always()`). Vérifiez sur le run suivant que le cache est bien restauré plutôt que reconstruit, et assurez-vous que sa clé dépend du contenu de `requirements.txt` pour ne jamais devenir obsolète silencieusement.

---

## Étape 7 – Statut CI obligatoire

### Consignes

Revenez sur la protection de branche configurée en séance 1, où `Require status checks to pass` avait volontairement été laissée décochée faute de CI existante. Maintenant qu'un pipeline tourne réellement, activez-la et sélectionnez précisément les checks requis – un check n'apparaît dans la liste que s'il a déjà tourné au moins une fois, il faut donc un run complet avant de revenir dans les réglages. Testez ensuite que la protection bloque effectivement un merge si la CI est rouge, en cassant volontairement un test dans une pull request dédiée, puis vérifiez que la correction débloque le merge sans intervention manuelle.

---

## Étape 8 – Badge & consolidation

### Consignes

Ajoutez le badge de statut CI dans le README pour rendre le pipeline visible immédiatement à quiconque arrive sur le dépôt – l'URL du badge suit un format standard basé sur `badge.svg`, mais elle doit correspondre exactement à l'organisation, au dépôt et au nom du fichier de workflow réels, sinon l'image reste cassée. Profitez-en pour documenter en une phrase ou deux ce que fait réellement le pipeline (déclencheurs, jobs, versions testées). Relisez l'ensemble des changements de la séance et vérifiez la checklist de livrable ci-dessous avant de considérer l'atelier terminé.

---

## ## Checklist de livrable (à vérifier avant de partir)

Pipeline CI GitHub Actions présent sur `main`, déclenché automatiquement sur push et sur pull request. Job `lint` (flake8) et job(s) `test` (pytest) bien séparés, avec `needs: lint`. Tests exécutés en matrice sur au moins 2 versions de Python. Cache des dépendances pip actif (visible via `Cache restored` sur un second run). Rapport de couverture conservé en artefact téléchargeable, y compris en cas d'échec (`if: always()`). Protection de branche sur `main` mise à jour pour exiger que les checks CI passent avant tout merge, testée avec une pull request volontairement cassée puis corrigée. Badge de statut CI visible et fonctionnel dans le README. Au moins un test ajouté par le groupe lui-même (`/status`), pas uniquement les tests fournis.

## ## En cas de blocage

Un workflow qui n'apparaît jamais dans l'onglet Actions : vérifier le chemin exact `.github/workflows/\*.yaml` (le `s` à `workflows` est obligatoire) et la validité du YAML (indentation, deux espaces cohérents). Un job qui échoue dès l'installation des dépendances : vérifier que `requirements.txt` est bien présent à la racine du dépôt et orthographié à l'identique dans la commande `pip install -r`. Une matrice qui semble tourner en parallèle mais utilise toujours la même version : vérifier que la version de Python demandée à l'étape d'installation n'est pas fixée en dur, mais bien reliée au paramètre de la matrice. Un cache qui ne s'active jamais (`Cache not found` à chaque run) : vérifier que la clé de cache dépend réellement du contenu du fichier de dépendances, et pas d'une valeur fixe qui ne changerait jamais. Une protection de branche qui ne propose aucun check à cocher : c'est normal tant qu'aucun run complet n'a eu lieu sur le dépôt – lancer un premier run, attendre qu'il se termine, puis revenir dans les réglages. Un badge cassé (icône d'image manquante) : l'URL du badge doit correspondre exactement à l'organisation et au dépôt réels, ainsi qu'au nom du fichier workflow, sensible à la casse.